

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 10

1 - Enunciado

Implemente novamente o estimador da tarefa 8 usando um contador compartilhado e o `rand_r` protegendo a soma de hits com `#pragma omp critical` e com `#pragma omp atomic`. Compare essas duas implementações com suas versões que usam contadores privados. Agora, compare estas com uma 5ª versão que utiliza apenas a cláusula `reduction` ao invés das diretivas de sincronização. Reflita sobre a aplicabilidade de desses mecanismos em termos de desempenho e produtividade e proponha um roteiro para quando utilizar qual mecanismo de sincronização, incluindo `critical` nomeadas e `locks` explícitos.

2 - Metodologia

Como já haviam 2 versões prontas foi necessário apenas atualizar as versões anteriores utilizadas na tarefa para a atual implementando as alterações descritas, a seguir foi compilado um a um os códigos da seguinte forma:

```
gcc -fopenmp parallel_share_critical.c -o share_critical  
export OMP_NUM_THREADS=12; time ./share_critical
```

```
gcc -fopenmp parallel_share_atomic.c -o share_atomic  
export OMP_NUM_THREADS=12; time ./share_atomic
```

```
gcc -fopenmp parallempi_atomic.c -o atomic  
export OMP_NUM_THREADS=12; time ./atomic
```

```
gcc -fopenmp parallempi_critical.c -o critical
```

```
export OMP_NUM_THREADS=12; time ./critical  
  
gcc -fopenmp parallel_reduction.c -o reduction  
export OMP_NUM_THREADS=12; time ./reduction
```

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

3.1 - parallel_share_critical.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <omp.h>  
#include <time.h>
```

```
int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

    double inicio = omp_get_wtime();
    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
    {
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) ^ tid;

        #pragma omp for lastprivate(ultimo)
        for(long long i = 0; i < N; i++){
            //Gear as coordenadas de x e y entre 0 e 1
            x = (double)rand_r(&seed)/RAND_MAX;
            y = (double)rand_r(&seed)/RAND_MAX;

            if ((x * x) + (y * y) <= 1.0)
            {
                #pragma omp critical
                pontos_dentro++;
            }
            ultimo = i;
        }
    }

    double fim = omp_get_wtime();
```

```

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

3.2 - parallel_share_atomic.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

    double inicio = omp_get_wtime();
    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
    {
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) ^ tid;
    }
}

```

```

#pragma omp for lastprivate(ultimo)

for(long long i = 0; i < N; i++){
    //Gear as coordenadas de x e y entre 0 e 1
    x = (double)rand_r(&seed)/RAND_MAX;
    y = (double)rand_r(&seed)/RAND_MAX;

    if ((x * x) + (y * y) <= 1.0)
    {
        #pragma omp atomic
        pontos_dentro++;
    }
    ultimo = i;
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

3.3 - parallelpi_critical.c

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <omp.h>

#include <time.h>

int main(){

    long long N = 10000000;

    long long pontos_dentro = 0;

    long long ultimo = -1;

    double x = 0.0,y = 0.0;

    double inicio = omp_get_wtime();

    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
    {

        long long local_dento_c = 0;

        int tid = omp_get_thread_num();

        unsigned int seed = (unsigned int)time(NULL) ^ tid;

        #pragma omp for lastprivate(ultimo)

        for(long long i = 0; i < N; i++){

            //Gear as coordenadas de x e y entre 0 e 1

            x = (double)rand_r(&seed)/RAND_MAX;

            y = (double)rand_r(&seed)/RAND_MAX;

            if ((x * x) + (y * y) <= 1.0)

            {

                local_dento_c++;

            }

            ultimo = i;

        }

    }
```

```

#pragma omp critical

pontos_dentro += local_dento_c;
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

3.4 - parallempi_atomic.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

```

```
double inicio = omp_get_wtime();

#pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
{
    long long local_dento_c = 0;

    int tid = omp_get_thread_num();

    unsigned int seed = (unsigned int)time(NULL) ^ tid;

    #pragma omp for lastprivate(ultimo)
    for(long long i = 0; i < N; i++){
        //Gear as coordenadas de x e y entre 0 e 1
        x = (double)rand_r(&seed)/RAND_MAX;
        y = (double)rand_r(&seed)/RAND_MAX;

        if ((x * x) + (y * y) <= 1.0)
        {
            local_dento_c++;
        }
        ultimo = i;
    }

    #pragma omp atomic
    pontos_dentro += local_dento_c;
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);
```

```

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

3.5 - parallel_reduction.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0, y = 0.0;

    double inicio = omp_get_wtime();
    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
    {
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) ^ tid;

        #pragma omp for lastprivate(ultimo) reduction(+:pontos_dentro)
        for(long long i = 0; i < N; i++){

```

```

//Gear as coordenadas de x e y entre 0 e 1
x = (double)rand_r(&seed)/RAND_MAX;
y = (double)rand_r(&seed)/RAND_MAX;

if ((x * x) + (y * y) <= 1.0)
{
    pontos_dentro++;
}
ultimo = i;
}
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

4 - Resultado

Para a resolução dessa tarefa foram feitos 5 testes cada um com uma versão do código que será apresentada a seguir através das figuras, o primeiro teste realizado foi utilizando a cláusula critical com variável compartilhada.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_10 _ □ X
> export OMP_NUM_THREADS=12; time ./share_critical share e critical.
Pontos sorteados: 10000000
Pontos dentro do círculo: 7853333
Valor estimado de Pi: 3.141333
Tempo Medido: 2.071188 (s)
./share_critical 24,06s user 0,00s system 1159% cpu 2,076 total
```

Figura 1: Saída share critical

Logo após foi executado o mesmo código com variável compartilhada porém usando o atomic e o resultado pode ser visto abaixo.:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_10 _ □ X
> export OMP_NUM_THREADS=12; time ./share_atomic share e critical.
Pontos sorteados: 10000000
Pontos dentro do círculo: 7852701
Valor estimado de Pi: 3.141080
Tempo Medido: 0.146887 (s)
./share_atomic 1,74s user 0,01s system 1143% cpu 0,152 total
```

Figura 2: Saída share atomic

Agora feitos os ajustes para que a variável que antes era compartilhada entre as threads seja privada de cada thread e utilizando o critical:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_10 _ □ X
> export OMP_NUM_THREADS=12; time ./critical
Pontos sorteados: 10000000
Pontos dentro do círculo: 7855960
Valor estimado de Pi: 3.142384
Tempo Medido: 0.041948 (s)
./critical 0,40s user 0,00s system 856% cpu 0,047 total
```

Figura 3: Saída critical

Agora atualizando o mesmo código usado pela versão com atomic em uma versão usando o atomic com variáveis privadas.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_10 _ □ X
> export OMP_NUM_THREADS=12; time ./atomic
Pontos sorteados: 10000000
Pontos dentro do círculo: 7855308
Valor estimado de Pi: 3.142123
Tempo Medido: 0.028406 (s)
./atomic 0,33s user 0,00s system 985% cpu 0,034 total
```

Figura 4: Saída atomic

E por fim utilizando a cláusula reduction e mantendo a mesma estrutura utilizando variáveis privadas para cada thread:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_10
> export OMP_NUM_THREADS=12; time ./reduction
Pontos sorteados: 10000000
Pontos dentro do círculo: 7854746
Valor estimado de Pi: 3.141898
Tempo Medido: 0.031810 (s)
./reduction 0,36s user 0,00s system 997% cpu 0,036 total
```

Figura 5: Saída reduction

Após aferir todos os experimentos de maneira individual, coletar os dados e printar as imagens do terminal de saída. Em seguida foi gerada a tabela 1, para a visualização mais organizadas dos dados:

Tabela 1: Dados coletados

Nome do Arquivo	Tempo de Execução	Resultado Obtido	Pontos dentro do Círculo
Share_critical	2,071188	3,141333	7853333
Share_atomic	0,146887	3,14108	7852701
Critical	0,041948	3,142384	7855960
atomic	0,028406	3,142123	7855308
Reduction	0,031810	3,141898	7854746

5 - Conclusão

O experimento põe em prática mais um dos principais pilares da programação paralela à sincronização. Essa se aplicada de maneira errada pode gerar excesso de carga nas threads. Logo cabe ao programador se atentar aos objetivos da sua implementação, pois códigos errados podem gerar resultados corretos.

Como esperado a pior versão para os testes realizados utiliza a cláusula critical com variáveis compartilhadas isso ocorre pois essa diretiva acaba criando serialização excessiva em ambos os testes e por consequência gera computação extra que pode ser observada

nos tempos $t1 \approx 2,07s$ versão com o contador compartilhado e $t3 \approx 0,042 s$ versão com o contador privado por thread. Em ambas as abordagens fica evidente que utilizar outra cláusula como o atomic ou reduction para a mesma estrutura de código por si só já resolveria esse problema.

A partir dos resultados, também é possível estabelecer um critério para a escolha do mecanismo de sincronização.

Quando a operação realizada se encaixa diretamente em uma redução, como a soma dos valores obtidos nessa tarefa, a cláusula reduction deve ser priorizada, pois além de ser uma implementação fácil que não exige código extra, permite que cada thread trabalhe com uma cópia local da variável e que os resultados sejam combinados ao final reduzindo a necessidade de sincronização durante o laço.

Quando é necessária a atualização de uma variável compartilhada por meio de operação simples e elementar como o incremento de um contador, o atomic é mais adequado pois como visto na tarefa, pois elas garantem a exclusão mútua na CPU sem o custo de bloqueio de threads.

Já a diretiva critical é mais indicada quando a região que precisa ser protegida possui mais de uma instrução ou quando a operação não pode ser expressa por uma atualização atômica. Quando existem diferentes regiões críticas independentes no programa, o uso de regiões critical nomeadas permite que regiões associadas a nomes diferentes possam ser executadas concorrentemente, evitando a serialização desnecessária de operações que protegem recursos independentes.

Por fim, o uso de locks explícitos, utilizando `omp_lock_t`, deve ser considerado quando é necessário um controle mais específico sobre a sincronização, como o gerenciamento de diferentes locks associados dinamicamente a recursos, ou quando as diretivas anteriores não oferecem o controle necessário. Apesar da maior flexibilidade, essa abordagem exige gerenciamento manual e aumenta a responsabilidade do programador.



Figura 6: Ordem de diretivas baseada nos testes